

Classes, Métodos e Construtores

Linguagem de Programação II

Universidade Estadual da Paraíba
Curso: Ciência da Computação

October 8, 2026

O que vamos aprender hoje?

- 1 Revisão rápida: classes e atributos
- 2 O que é **abstração** (e o que deixar de fora)
- 3 Diferença entre paradigma **estruturado** e **orientado a objetos**
- 4 **Métodos** dentro da classe e como a chamada funciona
- 5 **Sobrecarga** de métodos
- 6 **Construtores** e o que acontece na memória com `new`
- 7 **Sobrecarga** de construtores
- 8 Um exemplo completo, juntando tudo
- 9 *Prévia*: objetos que se comunicam entre si

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

Relembrando a Aula Passada

- Uma **classe** agrupa atributos relacionados, como um struct de C;
- Aluno tinha os atributos nome, idade e nota;
- `new` cria um **objeto** de verdade a partir da classe;
- Usamos `objeto.atributo` para ler ou alterar um valor.

Do Jeito que Aprendemos

```
1 public class Aluno {  
2     String nome;  
3     int idade;  
4     double nota;  
5 }
```

```
1 Aluno a1 = new Aluno();  
2 a1.nome = "Maria";  
3 a1.idade = 20;  
4 a1.nota = 8.5;
```

O que Está Faltando?

Nossa classe só **guarda dados**.

Mas no mundo real, um aluno também **faz coisas**:
se matricula, calcula sua média, é aprovado ou reprovado. . .

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

O que é Abstração?

😊 Definição: Abstração

É o processo de representar algo do mundo real dentro do programa, focando **só no que importa** para o problema que estamos resolvendo – e ignorando todo o resto de propósito.

Uma Analogia

Analogia

Pense num **carro**. Para dirigir, você usa o volante, o acelerador e o freio – sem precisar saber como o motor funciona por dentro. Você enxerga uma versão **simplificada** do carro, só com o que interessa para dirigir.

Um Aluno de Verdade tem MUITAS Informações...

Dados possíveis de um aluno:

- Nome
- Idade
- Nota
- CPF
- Endereço
- Altura e peso
- Cor do cabelo
- Time do coração
- Estilo musical favorito
- Número do calçado

Mas precisamos de tudo isso
no nosso programa?

Um Aluno de Verdade tem MUITAS Informações...

Dados possíveis de um aluno:

- Nome
- Idade
- Nota
- CPF
- Endereço
- Altura e peso
- Cor do cabelo
- Time do coração
- Estilo musical favorito
- Número do calçado

Mas precisamos de tudo isso
no nosso programa?

Depende do sistema
que estamos construindo!

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- **Sistema de notas da faculdade:** precisa de nome, matrícula, nota.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- Sistema de notas da faculdade: precisa de nome, matrícula, nota. **Cor do cabelo?**
Não.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- Sistema de notas da faculdade: precisa de nome, matrícula, nota. **Cor do cabelo? Não.**
- Academia de ginástica: precisa de nome, altura, peso, plano.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- Sistema de notas da faculdade: precisa de nome, matrícula, nota. Cor do cabelo? Não.
- Academia de ginástica: precisa de nome, altura, peso, plano. Time do coração? Não.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- **Sistema de notas da faculdade:** precisa de nome, matrícula, nota. **Cor do cabelo? Não.**
- **Academia de ginástica:** precisa de nome, altura, peso, plano. **Time do coração? Não.**
- **Biblioteca:** precisa de nome, número do cartão, livros emprestados.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- **Sistema de notas da faculdade:** precisa de nome, matrícula, nota. **Cor do cabelo? Não.**
- **Academia de ginástica:** precisa de nome, altura, peso, plano. **Time do coração? Não.**
- **Biblioteca:** precisa de nome, número do cartão, livros emprestados. **Nota acadêmica? Não.**

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- **Sistema de notas da faculdade:** precisa de nome, matrícula, nota. **Cor do cabelo? Não.**
- **Academia de ginástica:** precisa de nome, altura, peso, plano. **Time do coração? Não.**
- **Biblioteca:** precisa de nome, número do cartão, livros emprestados. **Nota acadêmica? Não.**
- **Loja de uniforme:** precisa de nome, tamanho de roupa, número do calçado.

Depende do Sistema que Estamos Construindo!

Vamos ver o mesmo aluno em sistemas diferentes:

- **Sistema de notas da faculdade:** precisa de nome, matrícula, nota. **Cor do cabelo? Não.**
- **Academia de ginástica:** precisa de nome, altura, peso, plano. **Time do coração? Não.**
- **Biblioteca:** precisa de nome, número do cartão, livros emprestados. **Nota acadêmica? Não.**
- **Loja de uniforme:** precisa de nome, tamanho de roupa, número do calçado. **CPF? Não (nesse caso).**

Tabela: O que Cada Sistema Precisa do Aluno

Dado	Sist. Notas	Academia	Biblioteca	Uniforme
Nome	✓	✓	✓	✓
Nota	✓	×	×	×
Altura / Peso	×	✓	×	✓
Livros empr.	×	×	✓	×
Tam. de roupa	×	×	×	✓
Cor do cabelo	×	×	×	×
Time do coração	×	×	×	×

Cor do cabelo e time do coração nunca importam – em nenhum desses sistemas!

A “Classe Aluno” Muda Conforme o Problema

Sistema de Notas

```
String nome  
String matricula  
double nota  
boolean aprovado()
```

Academia

```
String nome  
double altura  
double peso  
String plano
```

Biblioteca

```
String nome  
int numCartao  
int livrosEmpr  
boolean podePegar()
```

Não existe uma “classe Aluno perfeita” – existe a classe que resolve **o seu problema**.

Abstração na Classe Aluno

- Um aluno de verdade tem **centenas** de características;
- Para o nosso sistema de notas, só **nome, idade e nota** importam;
- Escolhemos representar só isso – o resto foi **deixado de fora** de propósito.

Dica

Abstrair é sempre uma **escolha**, e depende do problema. Não existe uma “classe Aluno perfeita” – existe a classe que resolve o **seu** problema. Inclua apenas os atributos que você vai **realmente usar** no sistema.

♡ Checkpoint

- Abstração = representar só o que importa, ignorando o resto;
- O que “importa” depende do **problema que o programa resolve**;
- Cor do cabelo e time do coração raramente entram em qualquer sistema acadêmico;
- Toda classe que criamos já é um exercício de abstração.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos**
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

Dois Jeitos de Organizar um Programa

Até agora, todo método que escrevemos era `static`, chamado direto do `main`. Isso vai mudar hoje – e essa mudança tem nome.

Paradigma Estruturado (ou Procedural)

😊 Definição: Paradigma Estruturado

O programa é organizado como uma sequência de **funções**, que recebem dados, processam e devolvem um resultado. Os **dados ficam separados** das funções que trabalham com eles.

É como funciona em C – e como vocês programaram até agora, com métodos `static`.

Exemplo: Jeito Estruturado

```
1 struct Aluno {  
2     string nome;  
3     float nota;  
4 };  
5  
6 int calcularSituacao(struct Aluno a) {  
7     if (a.nota >= 6.0) return 1; // aprovado  
8     return 0;                  // reprovado  
9 }
```

A função `calcularSituacao` é separada da struct – ela **recebe** o aluno como parâmetro.

Paradigma Orientado a Objetos

😊 Definição: Paradigma Orientado a Objetos

Os dados e as **ações relacionadas a eles** ficam agrupados dentro do mesmo objeto. O objeto passa a “saber fazer” coisas com os próprios dados.

É o jeito que vamos aprender a programar a partir de hoje.

Prévia: Jeito Orientado a Objetos

```
1 public class Aluno {  
2     String nome;  
3     double nota;  
4  
5     boolean calcularSituacao() {  
6         return nota >= 6.0;  
7     }  
8 }
```

Repare: `calcularSituacao` agora está **dentro** da classe, e usa `nota` diretamente – sem receber nada por parâmetro.

Comparando os Dois Paradigmas

Estruturado	Orientado a Objetos
Dados e funções separados	Dados e ações juntos, no objeto
Função recebe o dado como parâmetro	Objeto usa seus próprios dados
<code>calcularSituacao(aluno)</code>	<code>aluno.calcularSituacao()</code>

Um Detalhe Importante

Dica

Os dois paradigmas fazem parte de uma família maior chamada **imperativa**: em ambos, o programa é uma sequência de comandos que muda o estado dos dados, passo a passo. A diferença é só **como** dados e ações são organizados.

Checkpoint

♡ Checkpoint

- Estruturado: dados e funções separados – `funcao(dado);`
- Orientado a Objetos: dados e ações juntas – `objeto.acao();`
- A partir de agora, nossos métodos vão morar **dentro** da classe, sem `static`.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe**
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

Relembrando os Métodos static

- Até a aula passada, todo método era `public static`, chamado direto do `main`;
- Um método `static` pertence à **classe**, e não a nenhum objeto específico.

A partir de agora, vamos criar métodos que pertencem ao **objeto** – sem a palavra `static`.

O que é um Método (de Objeto)?

😊 Definição: Método

É uma **ação** que um objeto sabe executar. Diferente de uma função `static`, o método tem acesso **direto** aos atributos do próprio objeto – não precisa recebê-los por parâmetro.

static x sem static

static	sem static (método)
Pertence à classe	Pertence a cada objeto
Chamado direto: <code>nome(...)</code>	Chamado com objeto: <code>obj.nome(...)</code>
Não acessa atributos de objeto	Acessa os atributos do próprio objeto

Criando um Método Simples

```
1 public class Aluno {  
2     String nome;  
3     int idade;  
4     double nota;  
5  
6     void exibirDados() {  
7         System.out.println(nome);  
8         System.out.println(idade);  
9         System.out.println(nota);  
10    }  
11 }
```

Dentro do método, nome, idade e nota já estão disponíveis – são os atributos do **próprio objeto**.

Chamando o Método

```
1 Aluno a1 = new Aluno();  
2 a1.nome = "Maria";  
3 a1.idade = 20;  
4 a1.nota = 8.5;  
5  
6 a1.exibirDados();
```

Primeiro o **objeto**, depois o **ponto**, depois o **método** – igual fazíamos com atributos.

Como Ler uma Chamada de Método

`a1.exibirDados()`

a1

.

exibirDados()

o objeto

separador

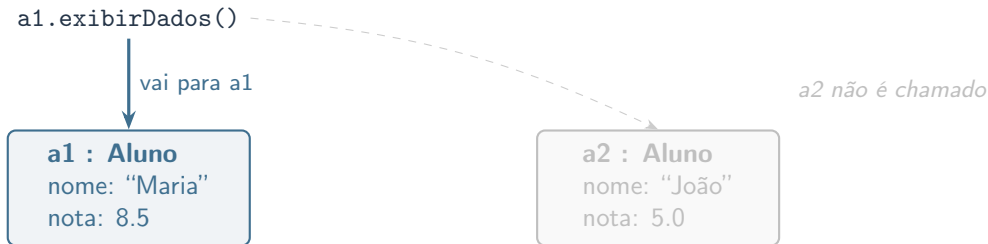
o método a executar

(quem faz a ação)

(o que vai ser feito)

“Vai ao objeto **a1** e pede para ele executar o método **exibirDados**.”

O Método Vai Até o Objeto Certo



O método usa **os dados de a1**. O objeto `a2` fica intocado.

Cada Objeto Tem Seus Próprios Dados

Antes: `a1.aumentarNota(1.0)`

a1 : Aluno
nota: **8.5**

a2 : Aluno
nota: **5.0**

Depois: `a1.aumentarNota(1.0)`

a1 : Aluno
nota: **9.5** ↑

a2 : Aluno
nota: **5.0** (intocado)

Alterar **a1** **nunca** afeta **a2**. Cada objeto tem sua própria cópia dos dados.

Método com Retorno

```
1 boolean calcularSituacao() {  
2     return nota >= 6.0;  
3 }
```

```
1 if (a1.calcularSituacao()) {  
2     System.out.println("Aprovado!");  
3 } else {  
4     System.out.println("Reprovado.");  
5 }
```

Método com Parâmetro

```
1 void aumentarNota(double valor) {  
2     nota = nota + valor;  
3 }
```

```
1 a1.aumentarNota(0.5); // nota de a1 aumenta em 0.5
```

O parâmetro `valor` vem de fora; `nota` continua sendo o atributo do objeto que chamou o método.

☹ Erro Comum

Tentar chamar o método **sem** um objeto na frente, como se ainda fosse `static`: `exibirDados()`; direto no `main` dá erro de compilação, porque o método agora pertence a um objeto – é preciso escrever `a1.exibirDados()`;

♡ Checkpoint

- Método (sem `static`) pertence ao objeto e acessa seus atributos diretamente;
- Chamamos com `objeto.metodo(...)`;
- Cada objeto tem seus **próprios dados** – alterar um não afeta o outro;
- Métodos podem ter parâmetros, retorno, ou nenhum dos dois.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos**
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

O que é Sobrecarga?

😊 Definição: Sobrecarga de Método

É ter **vários métodos com o mesmo nome** na mesma classe, desde que os **parâmetros** sejam diferentes – em quantidade ou em tipo.

Exemplo: Somar

```
1 int somar(int a, int b) {  
2     return a + b;  
3 }  
4  
5 double somar(double a, double b) {  
6     return a + b;  
7 }
```

Java escolhe automaticamente **qual** usar, olhando o tipo dos argumentos passados na chamada.

Chamando os Métodos Sobrecarregados

```
1 System.out.println(somar(2, 3));    // usa a versao int  
2 System.out.println(somar(2.5, 1.5)); // usa a versao double
```

☹ Erro Comum

Achar que mudar só o **tipo de retorno** já basta para sobrecarregar. Não basta! `int somar(int a, int b)` e `double somar(int a, int b)` têm os **mesmos parâmetros** – isso é erro de compilação, não sobrecarga.

Outro Exemplo: exhibirDados

```
1 void exhibirDados() {  
2     System.out.println(nome + " - " + nota);  
3 }  
4  
5 void exhibirDados(String prefixo) {  
6     System.out.println(prefixo + nome + " - " + nota);  
7 }
```

a1.exibirDados(); ou a1.exibirDados("Aluno: "); – a quantidade de parâmetros decide qual é chamado.

♡ Checkpoint

- Sobrecarga = mesmo nome, parâmetros diferentes;
- Java decide qual método usar pelos argumentos da chamada;
- Só mudar o tipo de retorno **não** é sobrecarga válida.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores**
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo

O Problema de Preencher Atributo por Atributo

```
1 Aluno a1 = new Aluno();  
2 a1.nome = "Maria";  
3 a1.idade = 20;  
4 a1.nota = 8.5;
```

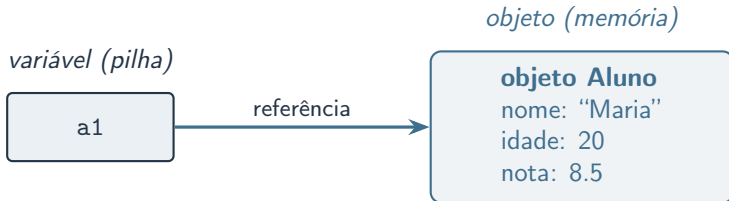
Repetitivo – e fácil de **esquecer** de preencher algum atributo.

O que é um Construtor?

😊 Definição: Construtor

É um método especial, chamado **automaticamente** quando usamos `new`, que serve para já criar o objeto **com os valores prontos**.

O que Acontece na Memória com `new`?



```
Aluno a1 = new Aluno(...);
```

`a1` não é o objeto – `a1` aponta para o objeto criado pelo `new` na memória.

Regras do Construtor

- Tem o **mesmo nome** da classe;
- **Não** tem tipo de retorno – nem `void`;
- É chamado automaticamente logo depois do `new`.

Um Primeiro Construtor

```
1 public class Aluno {  
2     String nome;  
3     int idade;  
4     double nota;  
5  
6     Aluno(String n, int i, double no) {  
7         nome = n;  
8         idade = i;  
9         nota = no;  
10    }  
11 }
```

Usando o Construtor

```
1 Aluno a1 = new Aluno("Maria", 20, 8.5);
```

Uma linha só – o objeto já nasce **completo**, com todos os atributos preenchidos.

E se os Parâmetros Tivessem o Mesmo Nome dos Atributos?

É natural querer nomear os parâmetros igual aos atributos, pra ficar mais claro: nome, idade, nota em vez de n, i, no.

Um Bug Sutil

```
1 Aluno(String nome, int idade, double nota) {  
2     nome = nome;  
3     idade = idade;  
4     nota = nota;  
5 }
```

☹ Erro Comum

Isso **não funciona!** Depois de criado, o objeto continua com os atributos vazios (null, 0, 0.0).

Por que Isso Acontece?

- Dentro do construtor, existem **dois** “nome”: o parâmetro e o atributo;
- Quando escrevemos só `nome`, o Java entende o **parâmetro** – ele está mais “perto”;
- O comando `nome = nome;` vira “parâmetro recebe o próprio parâmetro” – o atributo nunca é tocado.

Visualizando o this

nome = nome – Errado

nome (parâmetro)

🔄 recebe a si mesmo

nome (atributo) *nunca tocado*

this.nome = nome – Correto

nome (parâmetro)

↓ atributo recebe parâmetro

this.nome (atributo) ✓

A Solução: `this`

😊 Definição: `this`

`this` representa o **objeto atual**. `this.atributo` sempre se refere ao atributo do objeto – mesmo que exista um parâmetro com o mesmo nome.

Construtor Corrigido

```
1 Aluno(String nome, int idade, double nota) {  
2     this.nome = nome;  
3     this.idade = idade;  
4     this.nota = nota;  
5 }
```

Agora `this.nome` é o atributo, e `nome` (sem `this`) é o parâmetro – os dois lados fazem sentido.

Antes x Depois

Antes (bug)	Depois (com this)
<code>nome = nome;</code>	<code>this.nome = nome;</code>
Atributo continua vazio	Atributo recebe o parâmetro

Construtor Padrão

- Se você **não escrever nenhum** construtor, o Java cria um “vazio” (sem parâmetros) automaticamente – é o que já usávamos, `new Aluno()`;
- Mas cuidado: assim que você escreve **um** construtor próprio, esse construtor padrão desaparece.

Erro Comum

Esquecer de inicializar algum atributo dentro do construtor – ele fica com o valor padrão (0, false, null) sem você perceber.

Erro Comum

Errar o nome do construtor (não bater com o nome da classe) – nesse caso, o Java não entende como construtor, e vira só um método comum sem tipo de retorno, o que dá erro de compilação.

♡ Checkpoint

- Construtor tem o nome da classe e não tem tipo de retorno;
- `new` cria o objeto na memória; a variável guarda a referência;
- `this.atributo` garante que estamos falando do atributo, não do parâmetro;
- Escrever um construtor próprio remove o construtor padrão.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor**
- 8 Juntando Tudo

Por que Ter Mais de um Construtor?

Às vezes queremos criar um objeto com **menos** informações – por exemplo, só o nome, e preencher idade e nota depois.

😊 Definição: Sobrecarga de Construtor

Assim como métodos, uma classe pode ter **vários construtores**, desde que os parâmetros sejam diferentes.

Dois Construtores

```
1 Aluno(String nome) {  
2     this.nome = nome;  
3 }  
4  
5 Aluno(String nome, int idade, double nota) {  
6     this.nome = nome;  
7     this.idade = idade;  
8     this.nota = nota;  
9 }
```

Java Escolhe o Construtor Certo

```
1 Aluno a1 = new Aluno("Maria");  
2 Aluno a2 = new Aluno("Joao", 22, 7.0);
```

a1 usa o construtor de 1 parâmetro – idade e nota ficam com valor padrão (0 e 0.0). a2 usa o de 3 parâmetros.

♡ Checkpoint

- Vários construtores, mesma classe, parâmetros diferentes;
- Java escolhe automaticamente pelo que você passa em `new`;
- Atributos não preenchidos ficam com o valor padrão do tipo.

Sumário

- 1 Revisão Rápida
- 2 Abstração
- 3 Paradigma Estruturado x Orientado a Objetos
- 4 Métodos Dentro da Classe
- 5 Sobrecarga de Métodos
- 6 Construtores
- 7 Sobrecarga de Construtor
- 8 Juntando Tudo**

A Classe Aluno Completa – Parte 1

```
1 public class Aluno {  
2     String nome;  
3     int idade;  
4     double nota;  
5  
6     Aluno(String nome) {  
7         this.nome = nome;  
8     }  
9  
0     Aluno(String nome, int idade, double nota) {  
1         this.nome = nome;  
2         this.idade = idade;  
3         this.nota = nota;  
4     }
```

A Classe Aluno Completa – Parte 2

```
1 void exibirDados() {  
2     System.out.println(nome + " - " + idade + " anos - nota " + nota);  
3 }  
4  
5 boolean calcularSituacao() {  
6     return nota >= 6.0;  
7 }  
8  
9 void aumentarNota(double valor) {  
0     nota = nota + valor;  
1 }  
2 }
```

Usando a Classe no main

```
1 Aluno a1 = new Aluno("Maria", 20, 8.5);
2 Aluno a2 = new Aluno("Joao");
3
4 a1.exibirDados();
5 a2.exibirDados();
6
7 a2.aumentarNota(7.0);
8 a2.exibirDados();
9
10 if (a1.calcularSituacao()) {
11     System.out.println("Maria aprovada!");
12 }
```

Acompanhando o Código Passo a Passo

Linha executada	O que acontece	Estado de a2.nota
Aluno a2 = new Aluno("Joao");	cria a2 com 1 parâmetro	0.0 (padrão)
a2.exibirDados();	imprime Joao, 0 anos, nota 0.0	0.0
a2.aumentarNota(7.0);	nota = 0.0 + 7.0	7.0
a2.exibirDados();	imprime Joao, 0 anos, nota 7.0	7.0

Note: a1 e a2 são independentes – mudar um não afeta o outro.

Visualizando o Objeto Completo

objeto a1, do tipo Aluno

```
nome: "Maria"
```

```
idade: 20
```

```
nota: 8.5
```

```
métodos: exibirDados(), calcularSituacao(), aumentarNota()
```

Os métodos não ficam “guardados dentro” de cada objeto – mas cada chamada usa os dados **daquele** objeto específico.

Prévia: Objetos que se Comunicam

E se um método precisasse **receber outro objeto** como parâmetro?

```
1 boolean temNotaMaior(Aluno outro) {  
2     return this.nota > outro.nota;  
3 }
```

```
1 // no main:  
2 if (a1.temNotaMaior(a2)) {  
3     System.out.println("Maria tem nota maior que Joao!");  
4 }
```

Na próxima aula veremos como objetos colaboram entre si – passando outros objetos como parâmetro e retornando-os.

- ❶ Abstração: inclua só o que o **sistema precisa** – cor do cabelo raramente importa;
- ❷ Estruturado separa dados e função; OO junta dados e ação no objeto;
- ❸ Método (sem `static`) pertence ao objeto e acessa seus atributos diretamente;
- ❹ `a1.metodo()` – o objeto vem antes do ponto, o método depois;
- ❺ Cada objeto tem seus **próprios dados**: alterar `a1` nunca afeta `a2`;
- ❻ `new` cria o objeto na memória; a variável guarda a referência para ele;
- ❼ `this.atributo` resolve o conflito entre parâmetro e atributo de mesmo nome;
- ❽ Sobrecarga e sobrecarga de construtor: mesmo nome, parâmetros diferentes.